Evolutionary Computing

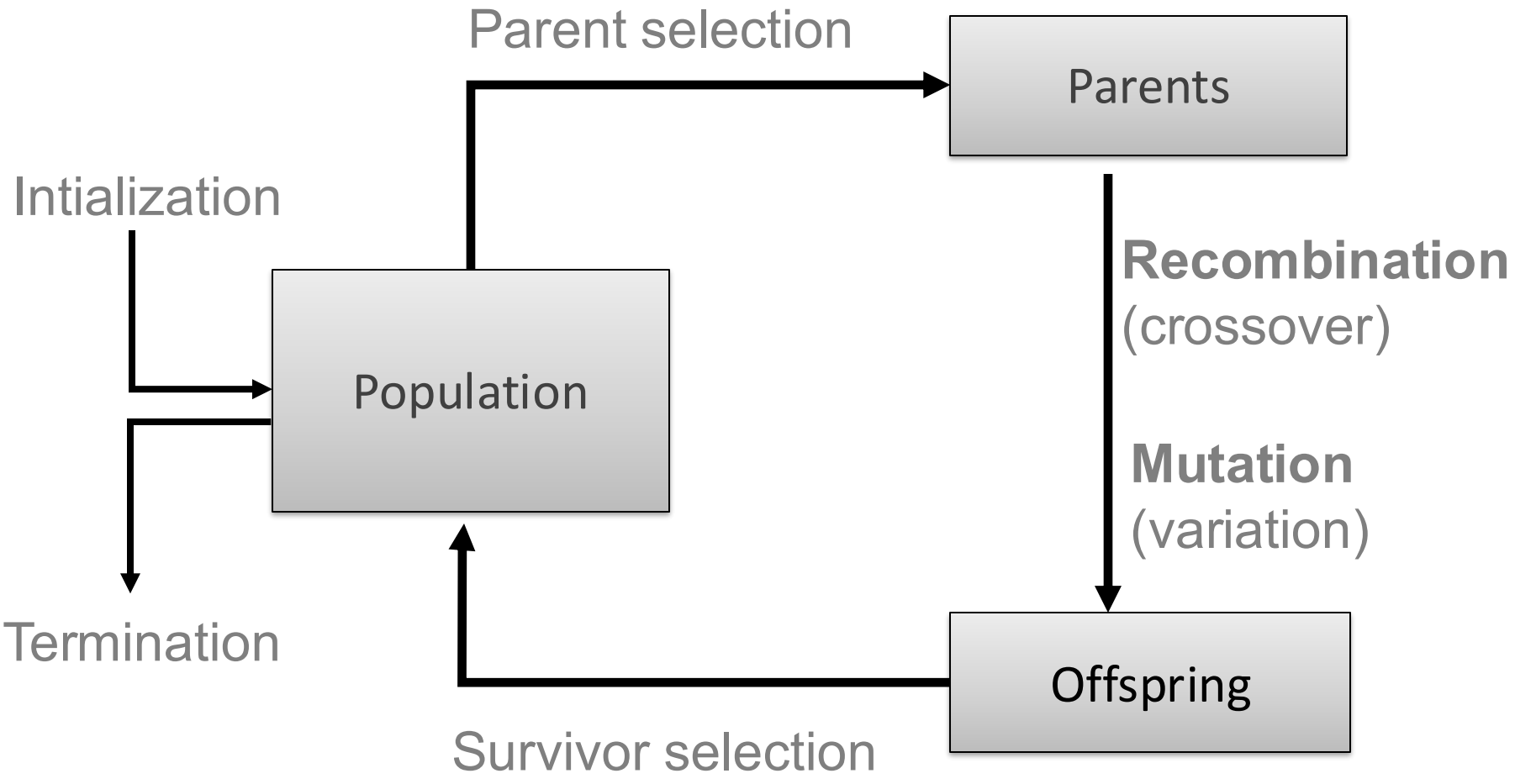
Chapter 4

Representation, Mutation, and Recombination

Chapter 4:
Representation, Mutation, and Recombination

- Role of **representation** and **variation** operators
- Most common **representation** of genomes:
  - Binary
  - Integer
  - Real-Valued or Floating-Point
  - Permutation
  - Tree

Scheme of an EA:
General scheme of EAs



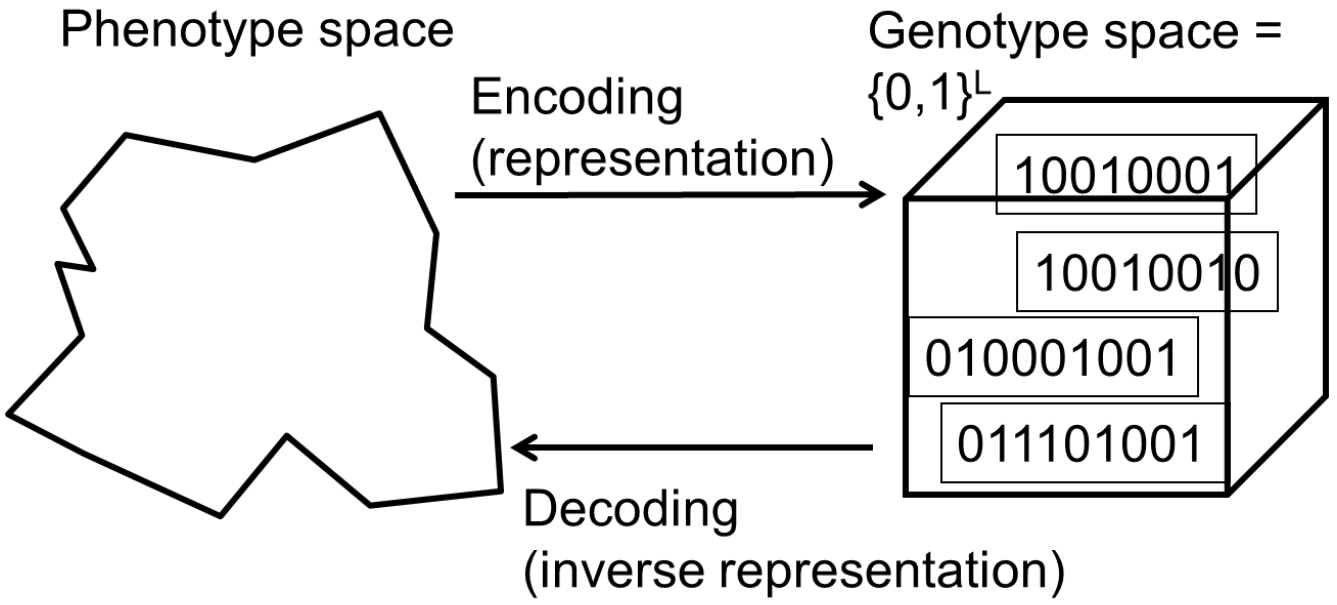
Role of representation and variation operators

- First stage of building an EA and **most difficult one**: choose ***right*** **representation** for the problem
- Variation operators: **mutation** and **crossover**
- **Type of variation operators** needed **depends** on chosen **representation**

- TSP problem
  - What are possible representations?

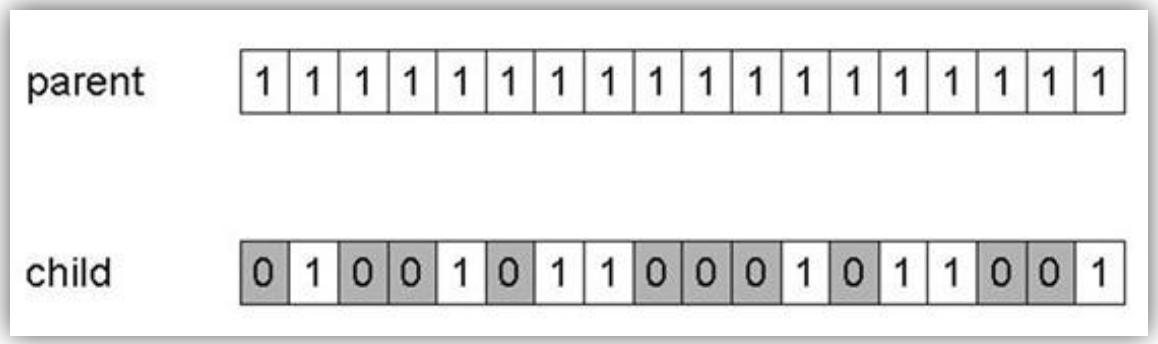
Binary Representation

- One of the earliest representations
- **Genotype** consists of a string of **binary digits**



Binary Representation: **Mutation**

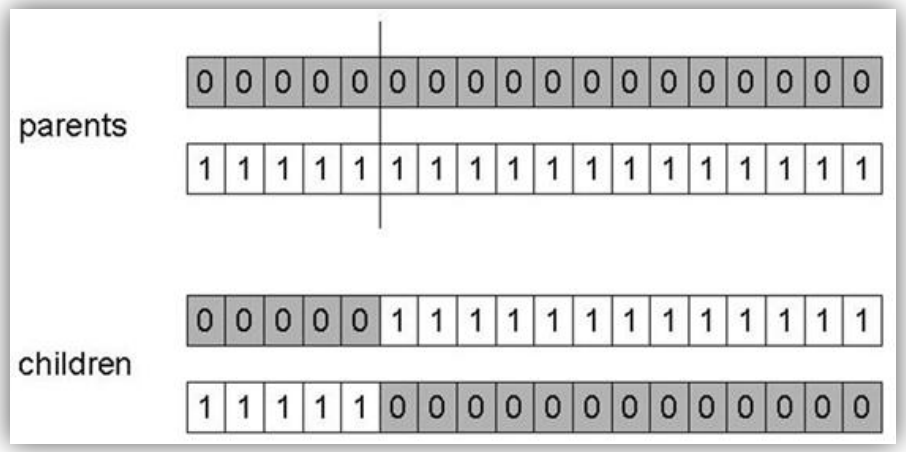
- Alter **each gene independently** with a probability $\mathbf{p_m}$
- p_m is called the **mutation rate**
  - Typically between **1/pop_size** and **1/ chromosome_length**



- Mutation can cause variable effect (use gray coding)

Binary Representation:
1-point crossover

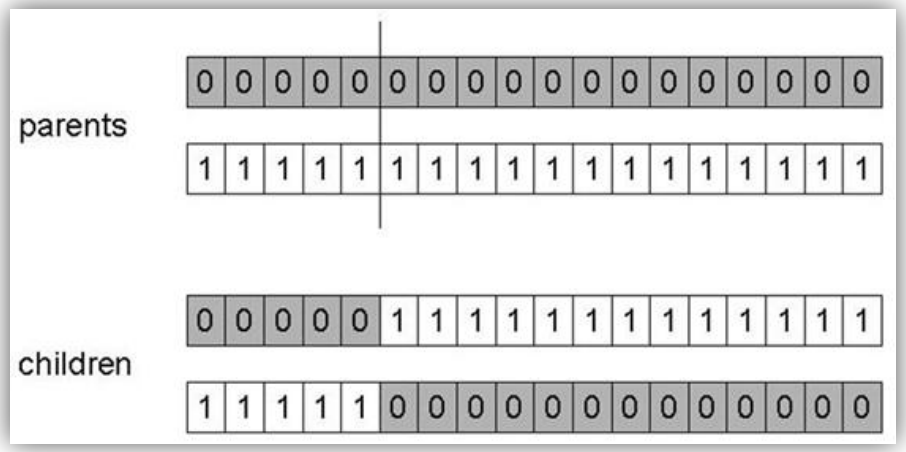
- Choose a **random point** on the **two parents**
- **Split parents** at this crossover point
- Create **children** by **exchanging tails**
- $\mathbf{P_c}$ (crossover probability) typically in range (0.6, 0.9). Controls **how often** crossover happens.
- When two parents are chosen for reproduction:
  - With probability **Pc**, you perform crossover (create new mixed offspring).
  - With probability **1 – Pc**, you **skip crossover** and simply copy the parents into the next generation (they survive unchanged).



P_c = 1 ... always crossover
High → more recombination, faster exploration
Low → more parental copying, slower exploration

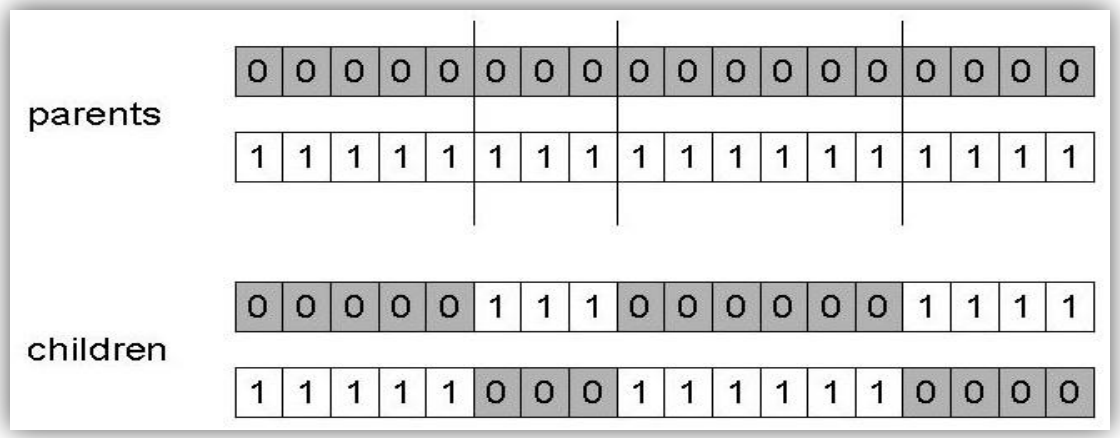
Binary Representation: Alternative Crossover Operators

- Why do we need other crossover(s)?
- Performance with 1-point crossover depends on the **order that variables occur in the representation**
  - More likely to **keep together genes** that are **near each other**
  - Can never keep together genes from opposite ends of string
  - This is known as **Positional Bias**
  - Can be exploited if we know about the structure of our problem, but this is not usually the case



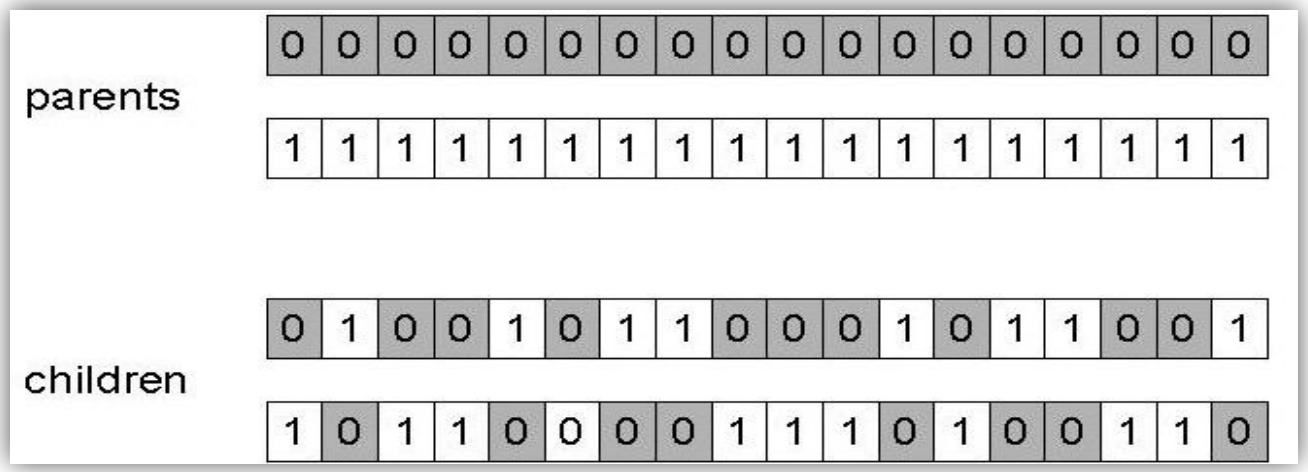
Binary Representation: n-point crossover

- Choose **n random crossover points**
- **Split** along those points
- **Glue parts**, **alternating** between parents
- Generalisation of 1-point (**still some positional bias**)



Binary Representation: Uniform crossover

- **Flip a coin for each gene** of the first child
- Assign **'heads' to one parent**, **'tails' to the other**
- Make an **inverse copy** of the gene for the **second child**
- Inheritance is independent of position



Binary Representation: Crossover OR mutation? (1/3)

- Decade long debate: which one is better / necessary / main-background

- Answer (at least, rather wide agreement):
  - it depends on the problem, but
  - in general, **it is good to have both**
  - both have another role
  - mutation-only-EA is possible, crossover-only-EA would not work

Binary Representation: Crossover OR mutation? (2/3)

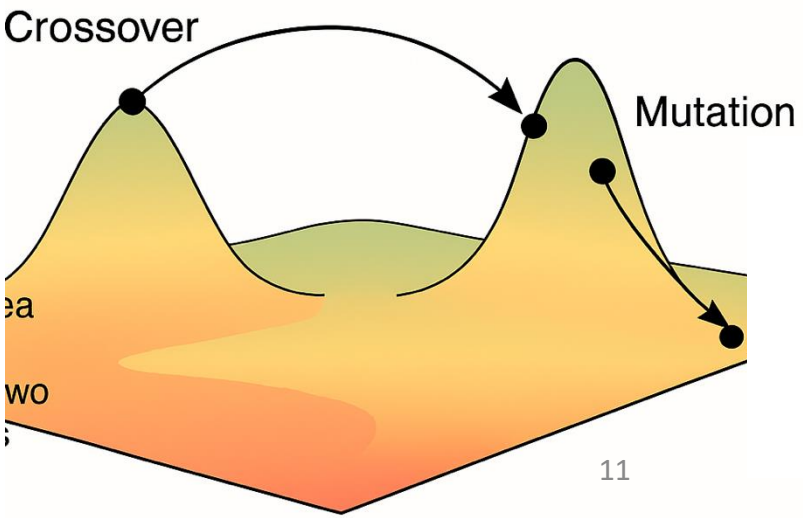
Exploration: Discovering promising areas in the search space, i.e. gaining information on the problem

Exploitation: Optimising **within a promising area**, i.e. using information

There is co-operation AND competition between them

• Crossover is **explorative**, it makes a ***big*** **jump** to an area somewhere "in between" two (parent) areas

• Mutation is **exploitative**, it creates **random *small* diversions**, thereby **staying near** (in the area of) the parent



Binary Representation: Crossover OR mutation? (3/3)

- Only **crossover** can **combine information from two parents**
- Only **mutation** can introduce **new information** (alleles)
- **Crossover** does not change the **allele frequencies** of the population
  (thought experiment: 50% 0's on first bit in the population, ?% after performing n crossovers)

Individual	Genes
1	**1** 0 0 1 1
2	**0** 1 0 1 0
3	**1** 1 1 0 0
4	**0** 0 0 1 1

At the **first bit position**, we have: 1010

P1 = **1 0** 0 1 1 Child1 = 1 0 0 1 0
P2 = 0 1 **0 1 0** Child2 = 0 1 0 1 1

- To hit the optimum you often need a **'lucky' mutation**

Integer Representation

- Nowadays it is generally accepted that it is **better to encode numerical variables directly** (integers, floating point variables)
- Some problems naturally have integer variables, e.g. **image processing parameters**
- Others take **categorical values** from a **fixed set** e.g. {blue, green, yellow, pink}

Integer Representaion

- **Same recombination** as for binary rep.
  - **N-point** / **uniform** crossover
- **Mutation**
  - **Random resetting** (esp. categorical variables)
    - With probability p_m a new value is chosen at random

Real-value Representation

- **Recombination**: Arithmetic, intermediate, BLX-α
- **Mutation:** "**creep**" i.e. more likely to move to similar value
  - **Adding a small (positive or negative) value** to each gene with probability p.

Real-Valued or Floating-Point Representation

1) What it is

- A solution (chromosome) is a **vector of real numbers**: $x = [x_1, x_2, ..., x_n]$.
- Each **gene** is a continuous variable in a range, e.g. $x_i \in [L_i, U_i]$.
- No encoding/decoding to bits. You work **directly** with real parameters.

Why: direct, precise, and natural for continuous optimisation.

2) Define the search space

- Set bounds per gene:
  Example: $x_1 \in [-5, 5]$, $x_2 \in [0, 10]$, ...

3) Initialisation

- Draw each gene **uniformly** from its bounds: $x_i \sim \mathcal{U}(L_i, U_i)$.

4) Fitness evaluation

- Compute objective $f(x)$.
- For minimisation, many EAs maximise $-f(x)$ or use ranks.

Example: $f(x) = x_1^2 + x_2^2$.
Fitness = $-f(x)$.

Real-Valued or Floating-Point Mutation

5) Mutation (core of real-valued EAs)

5.1 Gaussian (creep) mutation

- Small random change per gene with probability p_m:

$$x_i' = x_i + \sigma_i \cdot N(0,1)$$

- σ_i controls **step size** (exploration/exploitation).

Boundary handling:

- **Clip:** $x_i' \leftarrow \min(\max(x_i', L_i), U_i)$.
- Or **resample** if out of bounds.
- Or **reflect** back into range.

5.2 Self-adaptive mutation (ES idea)

- Evolve step sizes too:

$$\sigma_i' = \sigma_i \cdot \exp(\tau \cdot N(0,1))$$

  Then mutate x_i with new σ_i'.

- Adapts automatically to problem scale.

Counterpoint: needs careful parameters; can be slower to start.

Real-Valued or Floating-Point Representation: Crossover operators

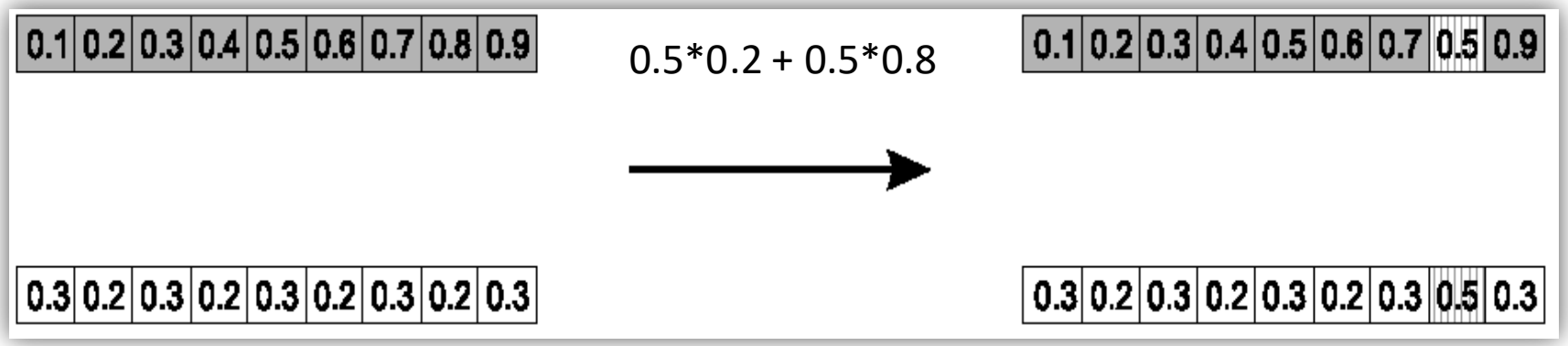
- **Discrete (same as for Integer)**:
  - each allele value in offspring *z* comes from one of its parents *(x,y)* with equal probability: $z_i = x_i$ or y_i
  - Could use n-point or uniform
- **Intermediate**
  - exploits idea of creating children "between" parents (hence a.k.a. *arithmetic* recombination)
  - $z_i = \alpha\, x_i + (1 - \alpha)\, y_i$ where α : $0 \leq \alpha \leq 1$.
  - The parameter α can be:
    - constant: uniform arithmetical crossover
    - variable (e.g. depend on the age of the population)
    - picked at random every time

Real-Valued or Floating-Point Representation: **Single** arithmetic crossover

- Parents: $\langle x_1,\ldots,x_n \rangle$ and $\langle y_1,\ldots,y_n \rangle$
- Pick a single gene (k) at random,
- $child_1$ is:

$$\langle x_1, ..., x_k, \alpha \cdot y_k + (1-\alpha) \cdot x_k, ..., x_n \rangle$$

- Reverse for other child. e.g. with α = 0.5

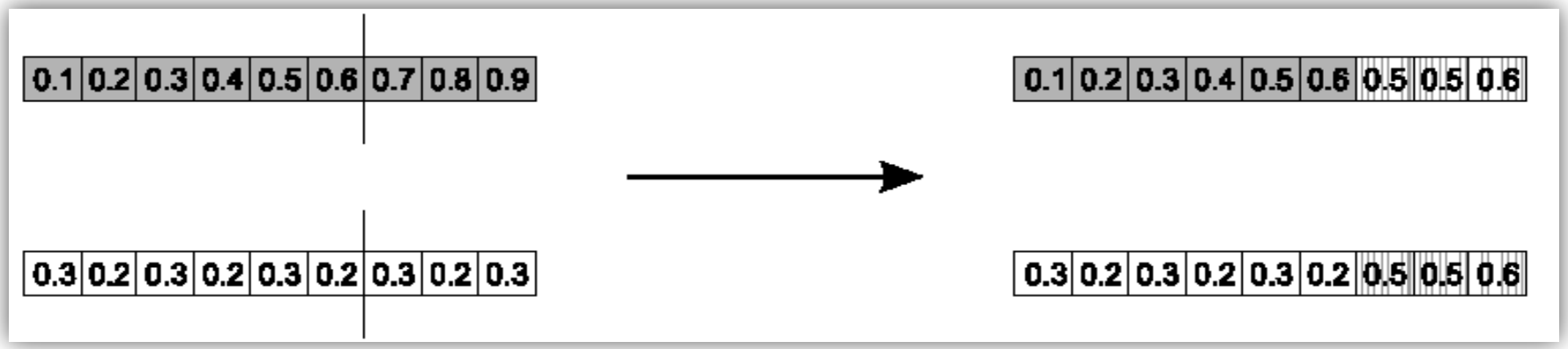


Real-Valued or Floating-Point Representation: **Simple** arithmetic crossover

- Parents: $\langle x_1,\ldots,x_n \rangle$ and $\langle y_1,\ldots,y_n \rangle$
- Pick a random gene (k) after this point mix values
- $child_1$ is:

$$\left\langle x_1,...,x_k,\alpha\cdot y_{k+1}+(1-\alpha)\cdot x_{k+1},...,\alpha\cdot y_n+(1-\alpha)\cdot x_n\right\rangle$$

- reverse for other child. e.g. with α = 0.5

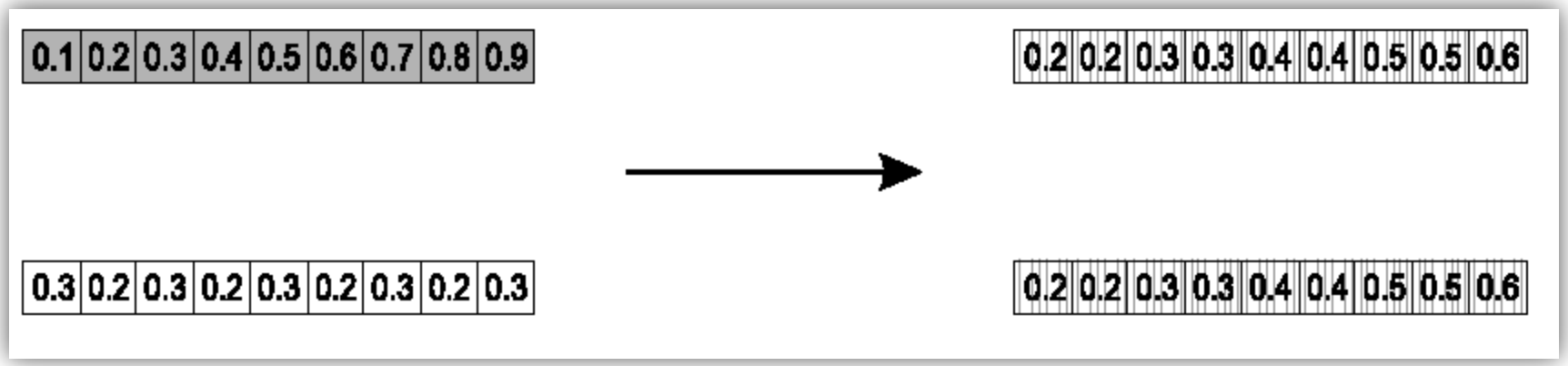


Real-Valued or Floating-Point Representation: **Whole** arithmetic crossover

- Most commonly used
- Parents: $\langle x_1,\ldots,x_n \rangle$ and $\langle y_1,\ldots,y_n \rangle$
- Child_1 is:

$$a \cdot \bar{x} + (1-a) \cdot \bar{y}$$

- reverse for other child. e.g. with $\alpha = 0.5$



Real-Valued or Floating-Point Representation: Blend Crossover – **BLX-α**

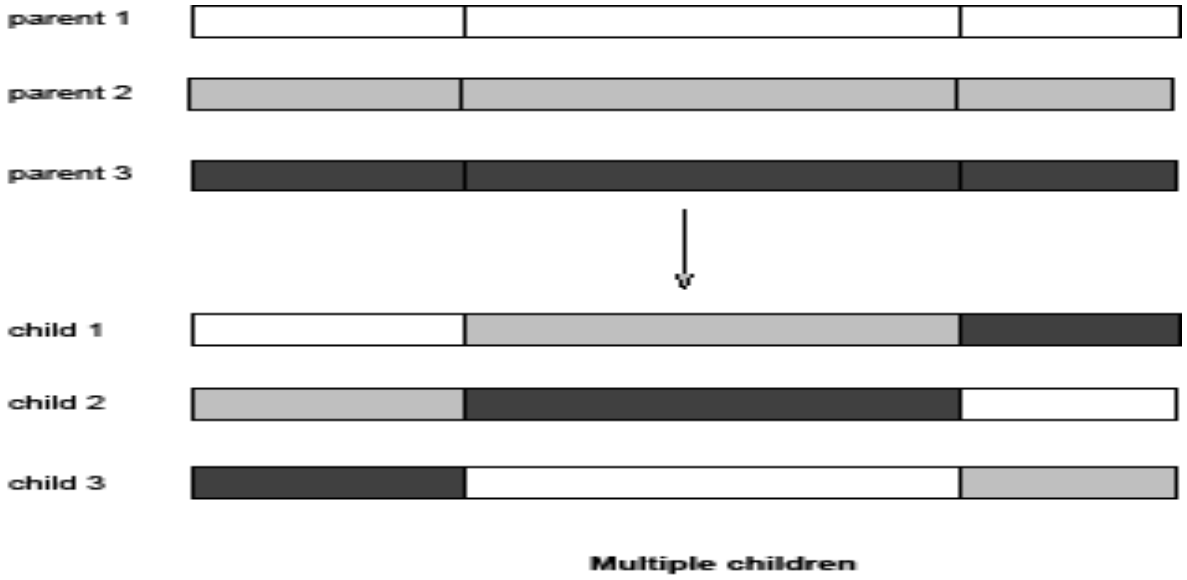
- It allows offspring to explore ***between*** *and* ***slightly beyond*** the parental values.
- Parents: $\langle x_1,\ldots,x_n \rangle$ and $\langle y_1,\ldots,y_n \rangle$
- Assume $x_i < y_i$
- $d_i = y_i - x_i$
- Random sample $z_i = [x_i - \alpha d_i, x_i + \alpha d_i]$

Real-Valued or Floating-Point Representation: **Multi-parent recombination**

- Recall that we are not constricted by the practicalities of nature
- Noting that mutation uses $n = 1$ parent, and "traditional" crossover $n = 2$, the extension to $n > 2$ is natural to examine
- Been around since 1960s, still **rare** but studies indicate useful

Real-Valued or Floating-Point Representation: Multi-parent recombination, type 1

- Idea: segment and recombine parents
- Example: diagonal crossover for *n* parents:
  - Choose *n-1* crossover points (same in each parent)
  - Compose *n* children from the segments of the parents in along a "diagonal", wrapping around



- This operator generalises 1-point crossover

Real-Valued or Floating-Point Representation: Multi-parent recombination, type 2

- Idea: arithmetical combination of (real valued) alleles
- Example: arithmetic crossover for *n* parents:
  - *i*-th allele in child is the average of the parents' *i*-th alleles

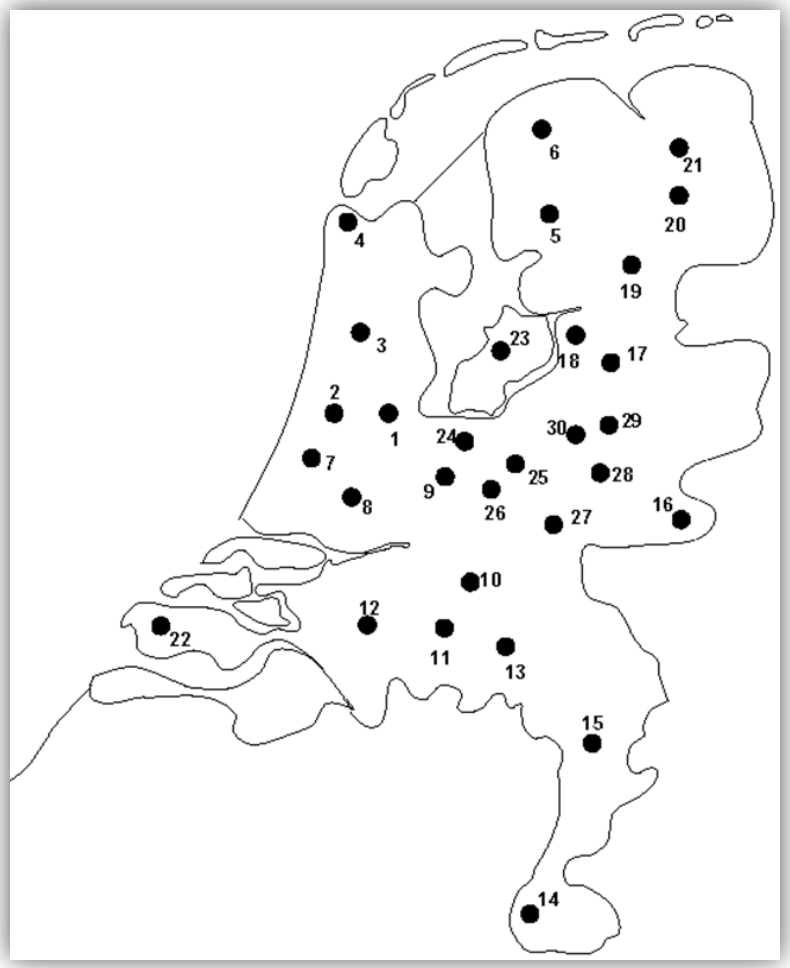
Permutation Representations

- **Ordering/sequencing** problems form a special type
- Task is (or can be solved by) **arranging** some objects in a certain **order**
  - Example: production **scheduling**: important thing is which elements are scheduled before others (order)
  - Example: **Travelling Salesman Problem** (TSP) : important thing is which elements occur **next** to each other (adjacency)
- These problems are generally expressed as a **permutation**:
  - if there are ***n* variables** then the representation is as a list of ***n* integers**, each of which occurs **exactly once**

Permutation Representation: TSP example

- Problem:
  - Given n cities
  - Find a complete tour with minimal length
- Encoding:
  - Label the cities 1, 2, … , *n*
  - One complete tour is one permutation (e.g. for n =4 [1,2,3,4], [3,4,2,1] are OK)
- Search space is BIG:

  for 30 cities there are 30! ≈ 10^{32} possible tours



Permutation Representations: Mutation

- Normal mutation operators lead to **inadmissible solutions**
  - e.g. bit-wise mutation: let gene *i* have value *j*
  - changing to some other value *k* would mean that *k* occurred twice and *j* no longer occurred
- Therefore must change **at least two values**
- Mutation parameter now reflects the **probability that some operator is applied once to the whole string**, rather than individually in each position

Permutation Representations: Swap mutation

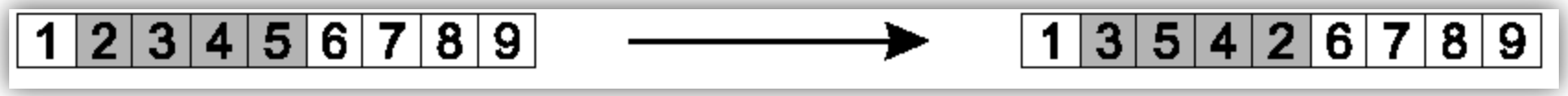
- Pick two alleles at random and swap their positions

Permutation Representations: Insert Mutation

- Pick **two allele** values at **random**
- Move the **second to follow the first**, **shifting** the rest along to accommodate
- Note that this preserves most of the order and the adjacency information

Permutation Representations: Scramble mutation

- Pick a subset of genes at random
- Randomly rearrange the alleles in those positions

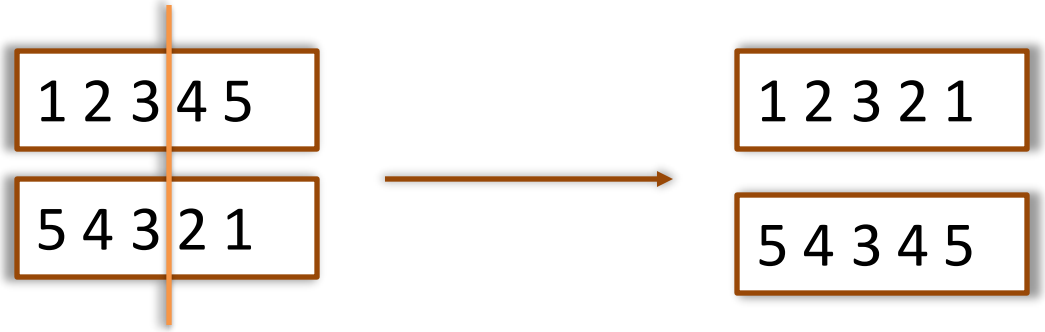


Permutation Representations: Inversion mutation

- Pick **two alleles** at **random** and then **invert the substring** between them.
- Preserves most adjacency information (only breaks two links) but disruptive of order information

Permutation Representations: Crossover operators

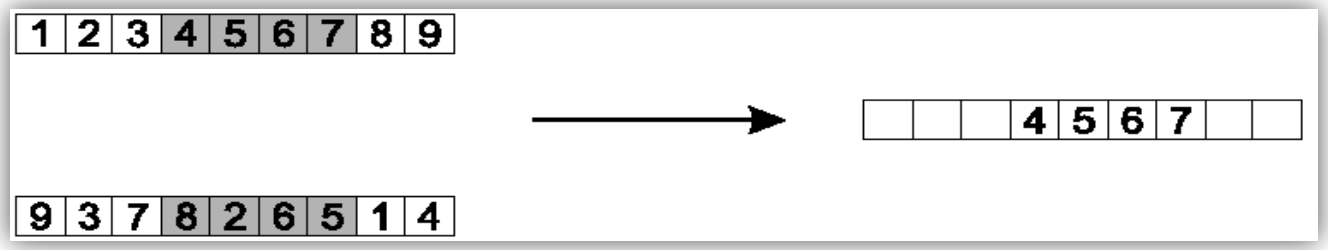
- "Normal" crossover operators **will often lead to inadmissible** solutions



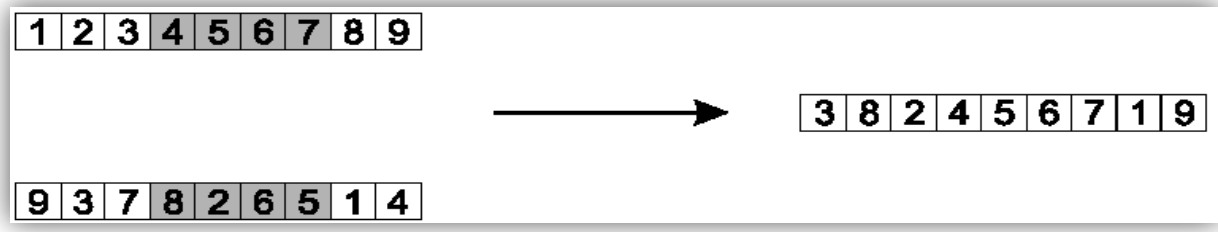
- Many specialised operators have been devised which focus on combining order or adjacency information from the two parents

Permutation Representations: Order 1 crossover

- Idea is to preserve relative order that elements occur
- Informal procedure:
  - 1. Choose an arbitrary part from the first parent
  - 2. Copy this part to the first child
  - 3. Copy the numbers that are not in the first part, to the first child:
    - starting right from cut point of the copied part,
    - using the **order** of the second parent
    - and wrapping around at the end
  - 4. Analogous for the second child, with parent roles reversed



Copy rest from second parent in order 1,9,3,8,2

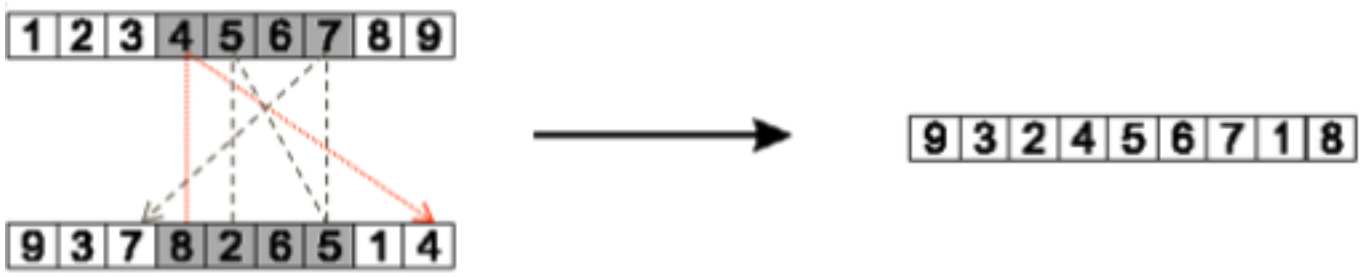


Permutation Representations: Partially Mapped Crossover (PMX) (1/2)

Informal procedure for parents P1 and P2:

- Select two crossover points (a and b).
- Copy segment P1[a:b] → C1[a:b], P2[a:b] → C2[a:b].
- Build mapping between swapped genes.
- Fill remaining positions using parent alleles and mapping to avoid duplicates.
- Produce two valid offspring.

Whenever we try to insert a value that is **already in the child**, we use the **mapping** to find a replacement that isn't duplicated.



Mapping
4 ↔ 8
5 ↔ 2
6 ↔ 6
7 ↔ 5

PyGad

Representation	Works "out of box" with PyGAD?	Comment
Binary	✅ Fully supported	use gene_space=[0,1]
Integer	✅ Fully supported	e.g. gene_space=range(0,10)
Real-valued	⚠ Partial	only replacement mutation, no blend crossovers
Permutation	❌ Unsupported	must encode as real-values (random keys) or write custom ops

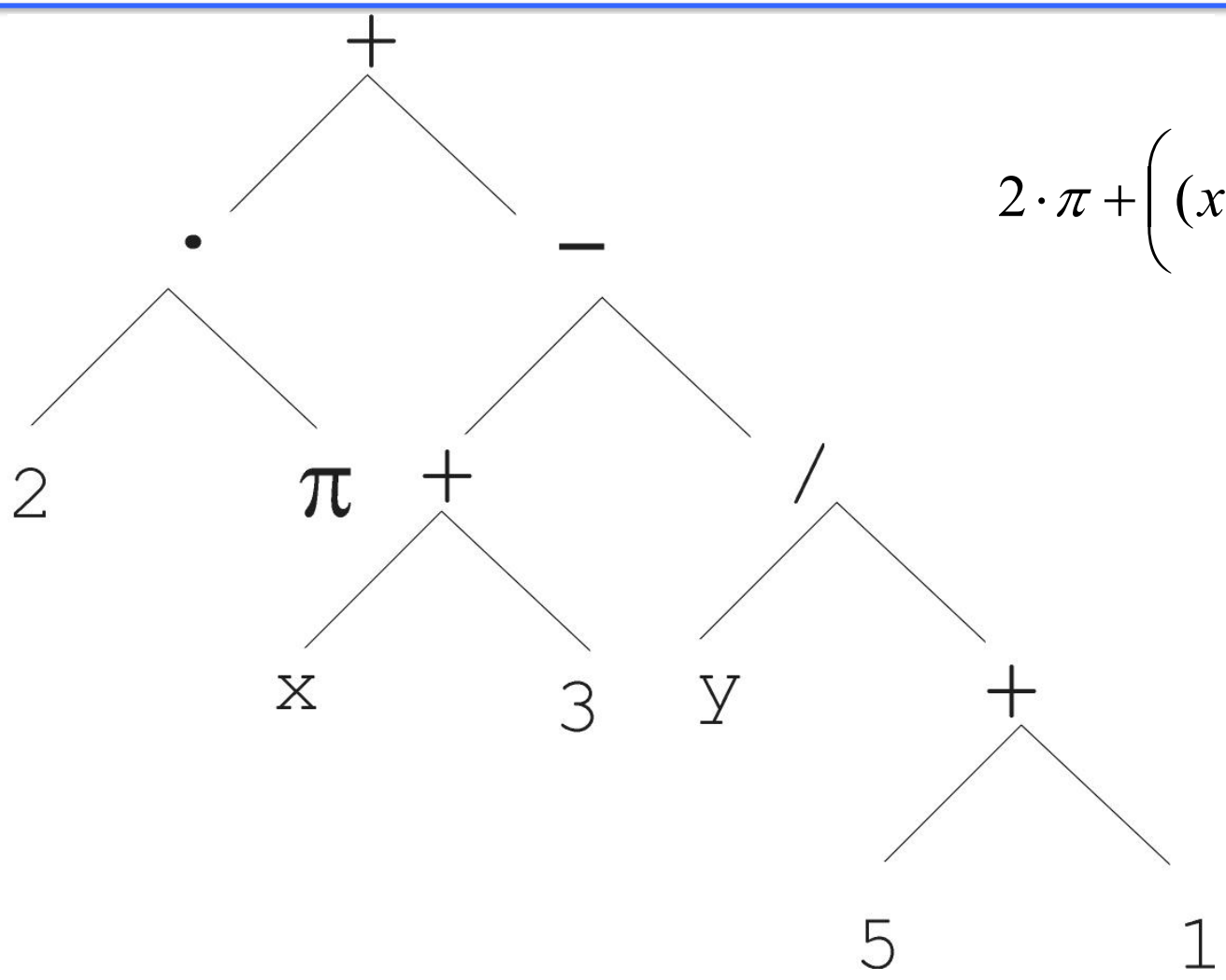
Representation	Operator Type	Specific Operator	Built-in in PyGAD?
Binary	Mutation	Standard bit flip / random replacement	✅ mutation_type='random'
	Crossover	1-point	✅ crossover_type='single_point'
		n-point (2-point)	✅ crossover_type='two_points'
		Uniform	✅ crossover_type='uniform' or 'scattered'
Integer	Mutation	Random resetting	✅ mutation_type='random'
	Crossover	(same as binary)	✅ Same as binary
Real-valued	Mutation	Creep / Gaussian / Uniform step	❌ Not built-in
	Crossover	Arithmetic (simple, single α)	❌ Not built-in
		Intermediate (blend + noise)	❌ Not built-in
		BLX-α	❌ Not built-in
Permutation	Mutation	Swap	❌ Not built-in
		Insert	❌
		Scramble	❌
		Inversion	❌
	Crossover	Order-1 (OX1)	❌
		PMX	❌

Tree Representation (1/6)

- Trees are a universal form, e.g. consider
- Arithmetic formula: $2 \cdot \pi + \left((x+3) - \frac{y}{5+1} \right)$
- Logical formula: $(x \wedge \text{true}) \rightarrow ((x \vee y) \vee (z \leftrightarrow (x \wedge y)))$
- Program:

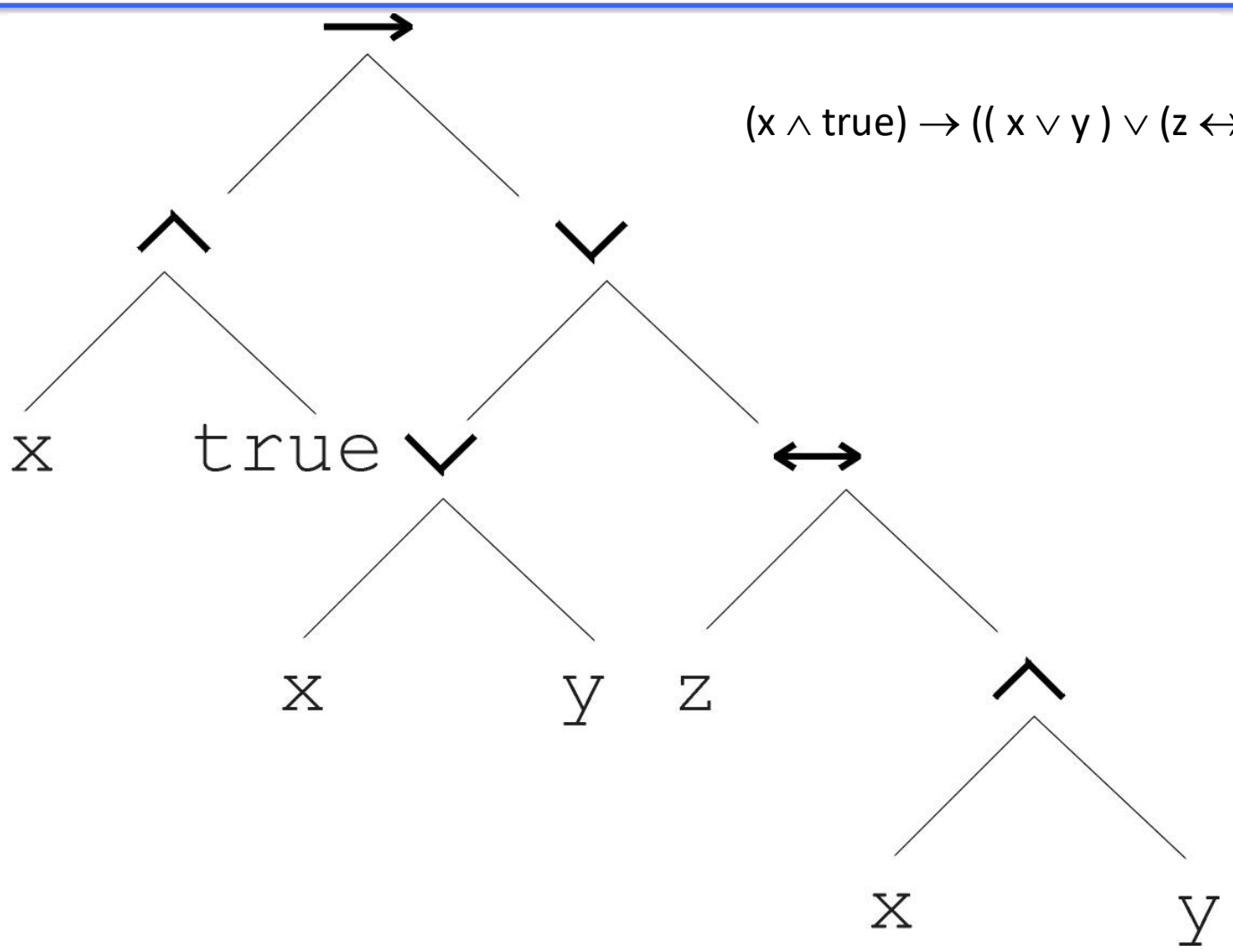
```
i =1;
while (i < 20)
{
        i = i +1
}
```

Tree Representation (2/6)



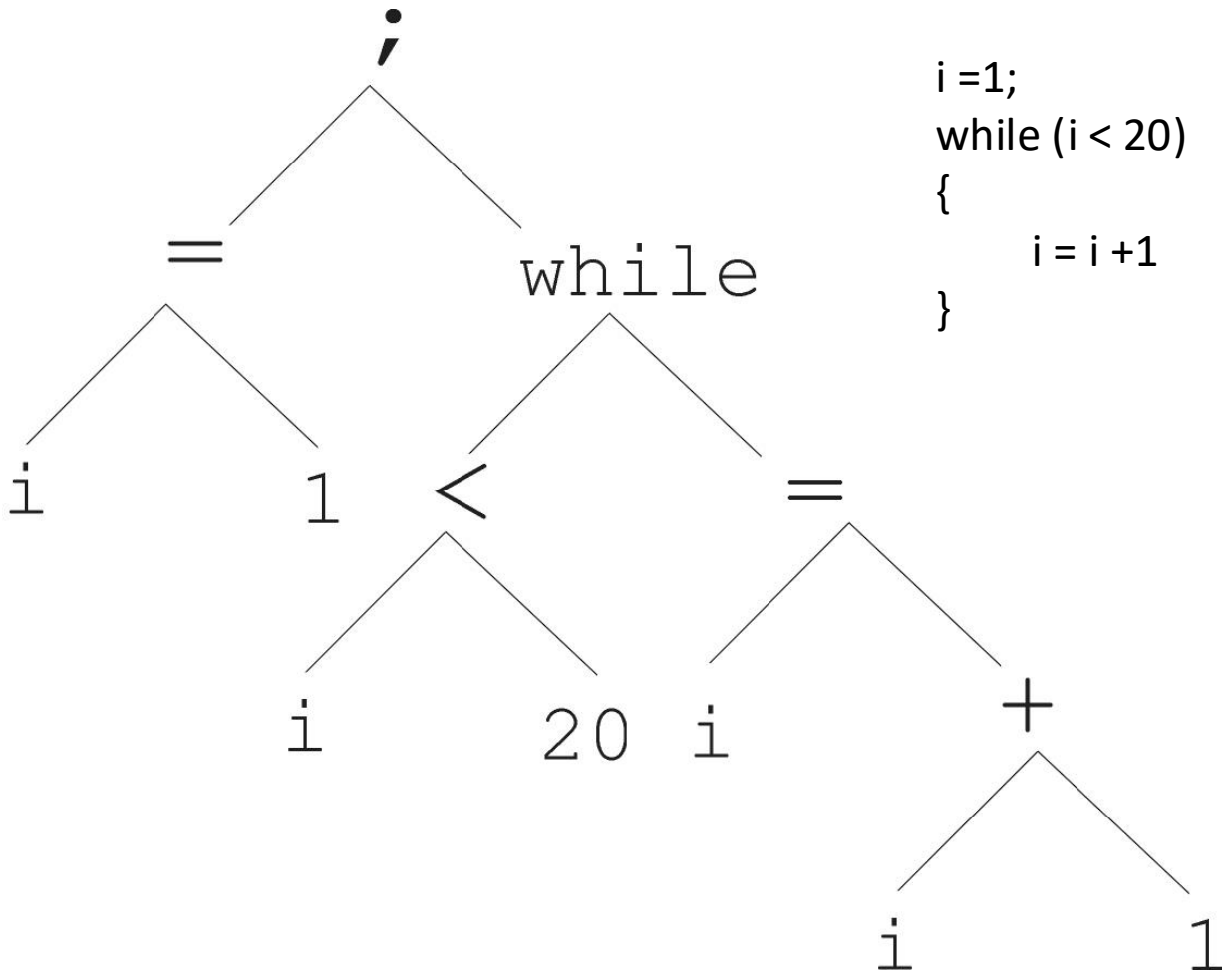
$$2 \cdot \pi + \left((x+3) - \frac{y}{5+1} \right)$$

Tree Representation (3/6)



$(x \wedge \text{true}) \rightarrow ((x \vee y) \vee (z \leftrightarrow (x \wedge y)))$

Tree Representation (4/6)




```
i =1;
while (i < 20)
{
      i = i +1
}
```

Tree Representation (5/6)

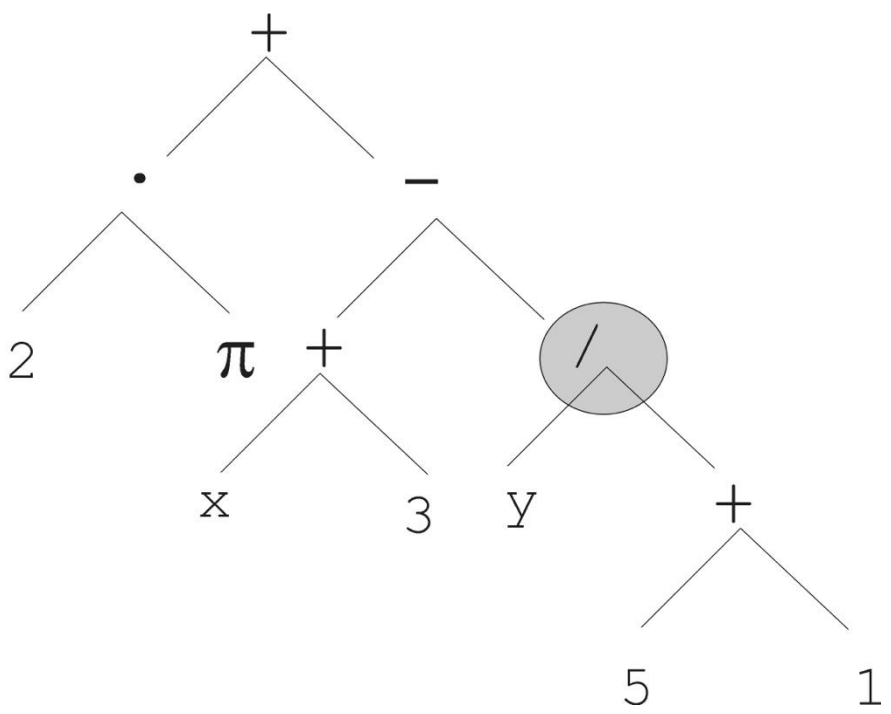
- In GA, ES, EP chromosomes are linear structures (bit strings, integer string, real-valued vectors, permutations)
- Tree shaped chromosomes are non-linear structures
- In GA, ES, EP the size of the chromosomes is fixed
- Trees in GP may vary in depth and width

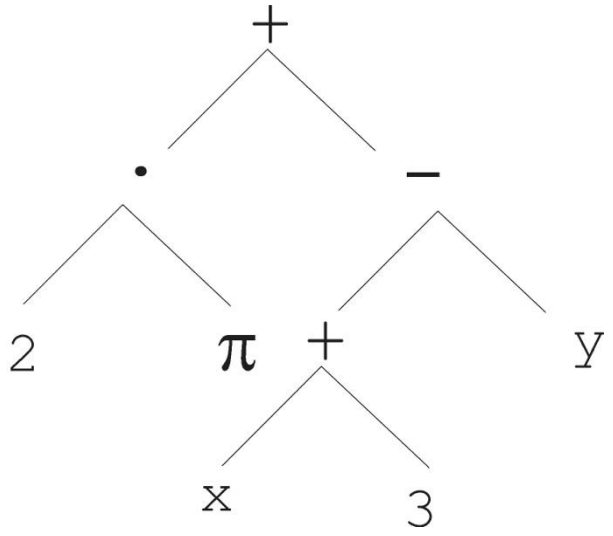
Tree Representation (6/6)

- Symbolic expressions can be defined by
  - Terminal set T
  - Function set F (with the arities of function symbols)
- Adopting the following general recursive definition:
  - Every $t \in T$ is a correct expression
  - $f(e_1, \ldots, e_n)$ is a correct expression if $f \in F$, arity(f)=n and $e_1, \ldots, e_n$ are correct expressions
  - There are no other forms of correct expressions
- In general, expressions in GP are not typed (closure property: any $f \in F$ can take any $g \in F$ as argument)

Tree Representation: Mutation (1/2)

- Most common mutation: replace randomly chosen subtree by randomly generated tree





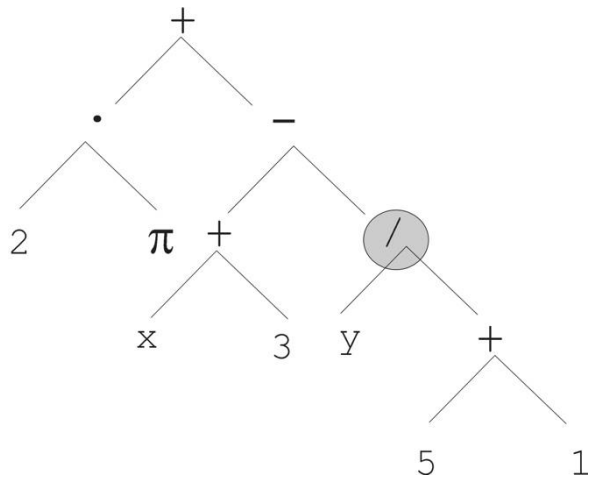
Tree Representation: Mutation (2/2)

- Mutation has two parameters:
  - Probability p_m to choose mutation
  - Probability to chose an internal point as the root of the subtree to be replaced
- Remarkably p_m is advised to be 0 (Koza'92) or very small, like 0.05 (Banzhaf et al. '98)
- The size of the child can exceed the size of the parent

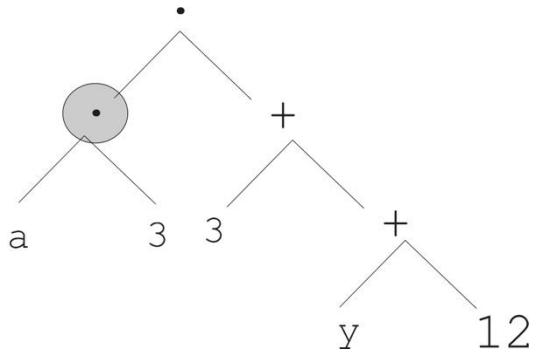
Tree Representation: Recombination (1/2)

- Most common recombination: exchange two randomly chosen subtrees among the parents
- Recombination has two parameters:
  - Probability p_c to choose recombination
  - Probability to chose an internal point within each parent as crossover point
- The size of offspring can exceed that of the parents

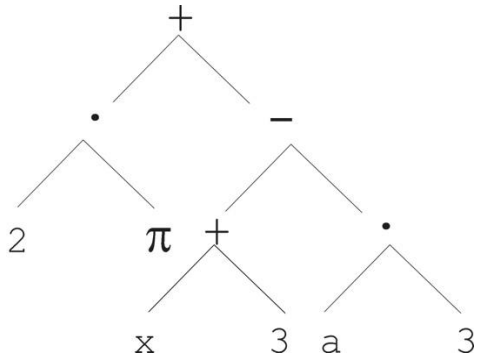
Tree Representation: Recombination (2/2)



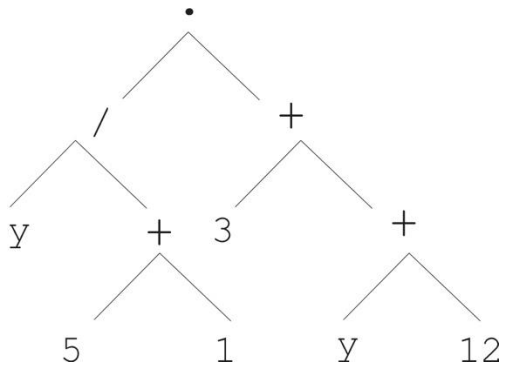
Parent 1



Parent 2



Child 1



Child 2